TaxonVision-MLOps

Comprehensive Technical Implementation Guide

Systems Architecture, Infrastructure, and Codebase Reference

Benjamin Förster

October 11, 2026

Abstract

This technical report serves as an exhaustive, implementation-level architectural manual and code base reference for the TaxonVision-MLOps platform. Designed to document every structural component, configuration file, deployment manifest, mathematical loss formulation, and runtime daemon in the active repository, this document provides complete visibility into the physical server topology (`hive-mind` running Ubuntu 26.04.1 LTS, NVIDIA Driver 615.71.09, CUDA 13.4, and the NVIDIA GeForce RTX 5070 Ti with Blackwell SM 120), the lightweight Kubernetes (k3s) cluster partitioned into `jenkins-ci` and `taxon-vision` namespaces, the Cloudflare Zero Trust ingress tunnel, and the four-slot NVIDIA GPU time-slicing device plugin. Furthermore, this manual dissects the pure Pixi multi-environment toolchain, the automated DagsHub storage remote for Data Version Control (DVC) and MLflow tracking, the Cui et al. Class-Balanced Loss dynamics on biological long-tail Pareto distributions, the Optuna Bayesian hyperparameter optimisation engine, Split Conformal Prediction, Energy-Based Out-of-Distribution rejection, and the unified FastAPI and HTMX operator dashboard. Every configuration parameter, file extension, and container specification is strictly analysed based upon active codebase evidence.

Contents

Abstract **i**

1 Repository Architecture, Directory Hierarchy, and File Extension Semantics **1**
1.1 Root Directory Topology 1
1.2 Configuration Manifests and Semantics 1
1.3 Specialised File Formats and Storage Conventions 1

2 Bare-Metal Server Environment and Pixi Toolchain Interlock **2**
2.1 Physical Compute Node Baseline 2
2.2 Multi-Target Toolchain and Pixi Environment Isolation 2
2.3 Command Orchestration via Justfile 2

3 Lightweight Kubernetes (k3s) Cluster Topology and Security RBAC **3**
3.1 Cluster Architecture and Network Edge 3
3.2 Namespace Partitioning and Multi-Tenant Segregation 3
3.3 Role-Based Access Control (RBAC) Specification 3

4 GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas **4**
4.1 The Single-GPU Multi-Tenancy Challenge 4
4.2 NVIDIA Device Plugin Time-Slicing Configuration 4
4.3 Deterministic VRAM Budget Capping and OOM Prevention 4

5 Containerisation Architecture: Dockerfile and Composition Manifests **5**
5.1 Multi-Stage Container Construction Strategy 5
5.2 Auxiliary Services and Docker Compose Topology 5
5.3 Container Hardening and Privilege Minimisation 5

6 Kubernetes Workloads, Service Manifests, and Rolling Updates **6**
6.1 API Deployment Specification 6
6.2 Health Probes and Zero-Downtime Rolling Restarts 6
6.3 Kubernetes Service Abstraction and Routing 6

7 Declarative Jenkins Pipeline and Ephemeral Kubernetes Agents **7**
7.1 Declarative Pipeline Architecture and Pod Template 7
7.2 Ephemeral Agent Lifecycle and Cache Volume Mounting 7
7.3 Stage-by-Stage Execution and Promotion Gates 7

8 Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits **8**
8.1 iNaturalist API Ingestion and Query Mechanics 8
8.2 Zero-Copy Apache Arrow Streaming Pipeline 8
8.3 Open License Verification and Attribution Retention 8

9 Data Version Control (DVC) and Remote Storage Topology **9**
9.1 DVC Pipeline Graph and Cryptographic Manifests 9

9.2 DagsHub S3-Compatible Remote Storage Architecture 9
9.3 Synchronization Lifecycles in CI/CD and Local Workstations 9

10 Domain Modelling, DarwinCore Standards, and Taxonomic Trees 10
10.1 DarwinCore Taxon Schema and Catalog Implementation 10
10.2 Observation Entities and Spatiotemporal Metadata 10
10.3 License Verification Domain Invariants 10

11 Vision Backbone Factory and Feature Caching Engine 11
11.1 Backbone Factory Architecture 11
11.1.1 Candidate Foundation Models 11
11.1.2 Parameter Freezing and Dimensional Introspection 12
11.2 Feature Extraction and Embedding Pre-Computation 12
11.2.1 The Decoupled Extraction Paradigm 12
11.2.2 Batch Extraction Mechanics and Tensor Normalisation 13
11.3 Persistent Embedding Caching System 13
11.3.1 On-Disk Serialisation and Key Generation 13
11.3.2 Memory-Mapped Tensor Loading 13
11.4 Throughput and Resource Benchmarking 14
11.4.1 Complexity and Speedup Analysis 14

12 Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics 15
12.1 The Biological Long-Tail Pareto Distribution 15
12.2 Effective Number of Samples Mathematical Derivation 15
12.3 PyTorch Implementation and Numerical Stabilisation 15

13 Classification Head Architecture, Optimization Loops, and Callbacks 16
13.1 Classification Head Geometry and Transfer Learning 16
13.2 Cached Embedding Training Loop and Mini-Batch Dynamics 16
13.3 Convergence Callbacks and Best-Weight Restoration 16

14 Bayesian Hyperparameter Optimization with Optuna TPE and ASHA 17
14.1 Bayesian Optimization Principles and TPE Mechanics 17
14.2 Search Space Parameterisation and Objective Formulation 17
14.3 Asynchronous Successive Halving (ASHA) Pruning in Practice 17

15 Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry 18
15.1 Experiment Tracking Architecture on DagsHub 18
15.2 Deterministic Lifecycle Management via RAII Scopes 18
15.3 Cryptographic Lineage Capture and Automated Model Promotion 18

16 Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection 19
16.1 The Epistemic Safety Problem in Biological Vision 19
16.2 Split Conformal Prediction and Statistical Coverage Bounds 19
16.3 Helmholtz Free Energy Out-of-Distribution Detection 19

17 Active Learning Strategy and Priority Triage Queue 20
17.1 Uncertainty-Based Query Acquisition Mechanics 20
17.2 Priority Triage Queue Implementation 20
17.3 Closing the Continuous Retraining Annotation Loop 20

18 ONNX Export, Zero-Allocation Runtime, and INT8 Quantization 21
18.1 Computational Graph Export and Dynamic Axes 21
18.2 Zero-Allocation C++ Inference Execution Engine 21
18.3 INT8 Quantization Calibration and Class Activation Mapping 21

19 FastAPI REST Service and Server-Rendered HTMX Operator Dashboard 22

19.1 Unified REST and Web Application Architecture 22
19.2 API Schemas, Endpoints, and Inference Routing 22
19.3 Server-Rendered HTMX Operator Interface 22

20 Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection 23
20.1 Prometheus Telemetry Instrumentation and Metrics Design 23
20.2 Serving Latency and Conformal Set Monitoring 23
20.3 Embedding Representation Drift via Wasserstein and MMD 23

21 Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets 24
21.1 Testing Architecture and Test Matrix Topology 24
21.2 Mathematical Invariant Assertions and Property-Based Testing 24
21.3 Cognitive Complexity Budgets and Pre-Commit Quality Gates 24

22 Open Knowledge Format (OKF v0.2) and Master Agent Routing 25
22.1 Open Knowledge Format (OKF v0.2) Schema and Trust Tiers 25
22.2 Multi-Agent Governance and Decision Routing Matrices 25
22.3 Automated Validation Scripts and D3 Knowledge Graph Visualization 25

23 Developer Operations, Environment Setup, and Engineering Runbooks 26
23.1 Developer Workstation Onboarding and Environment Setup 26
23.2 Dataset Synchronization and Remote Storage Runbook 26
23.3 Cluster Operations, GPU Debugging, and Emergency Rollback 26

Chapter 1

Repository Architecture, Directory Hierarchy, and File Extension Semantics

1.1 Root Directory Topology

1.2 Configuration Manifests and Semantics

1.3 Specialised File Formats and Storage Conventions

Chapter 2

Bare-Metal Server Environment and Pixi Toolchain Interlock

2.1 Physical Compute Node Baseline

2.2 Multi-Target Toolchain and Pixi Environment Isolation

2.3 Command Orchestration via Justfile

Chapter 3

Lightweight Kubernetes (k3s) Cluster Topology and Security RBAC

3.1 Cluster Architecture and Network Edge

3.2 Namespace Partitioning and Multi-Tenant Segregation

3.3 Role-Based Access Control (RBAC) Specification

Chapter 4

GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas

4.1 The Single-GPU Multi-Tenancy Challenge

4.2 NVIDIA Device Plugin Time-Slicing Configuration

4.3 Deterministic VRAM Budget Capping and OOM Prevention

Chapter 5

Containerisation Architecture: Dockerfile and Composition Manifests

5.1 Multi-Stage Container Construction Strategy

5.2 Auxiliary Services and Docker Compose Topology

5.3 Container Hardening and Privilege Minimisation

Chapter 6

Kubernetes Workloads, Service Manifests, and Rolling Updates

6.1 API Deployment Specification

6.2 Health Probes and Zero-Downtime Rolling Restarts

6.3 Kubernetes Service Abstraction and Routing

Chapter 7

Declarative Jenkins Pipeline and Ephemeral Kubernetes Agents

7.1 Declarative Pipeline Architecture and Pod Template

7.2 Ephemeral Agent Lifecycle and Cache Volume Mounting

7.3 Stage-by-Stage Execution and Promotion Gates

Chapter 8

Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits

8.1 iNaturalist API Ingestion and Query Mechanics

8.2 Zero-Copy Apache Arrow Streaming Pipeline

8.3 Open License Verification and Attribution Retention

Chapter 9

Data Version Control (DVC) and Remote Storage Topology

9.1 DVC Pipeline Graph and Cryptographic Manifests

9.2 DagsHub S3-Compatible Remote Storage Architecture

9.3 Synchronization Lifecycles in CI/CD and Local Workstations

Chapter 10

Domain Modelling, DarwinCore Standards, and Taxonomic Trees

10.1 DarwinCore Taxon Schema and Catalog Implementation

10.2 Observation Entities and Spatiotemporal Metadata

10.3 License Verification Domain Invariants

Chapter 11

Vision Backbone Factory and Feature Caching Engine

The operational deployment of fine-grained biological classifiers in production environments necessitates a rigorous equilibrium between predictive accuracy and computational efficiency. Deep neural networks, specifically those leveraging high-capacity attention mechanisms or densely parameterized convolutions, impose substantial computational burdens during both training and inference. To mitigate these constraints while maximising adaptability across diverse taxonomic domains, the TaxonVision architecture formally decouples the extraction of robust visual representations from the subsequent linear classification phase. This chapter mathematically details the architectural abstractions underlying the foundation model factory, the mechanics of representation extraction, the engineering invariants governing the persistent embedding caching engine, and the resulting multi-objective performance optimisations.

11.1 Backbone Factory Architecture

At the core of the platform's vision pipeline is the `BackboneFactory` abstraction, instantiated via the `src/taxon_vision/models/factory.py` module. This module provides a unified, type-safe interface for instantiating diverse foundational vision architectures through the `timm` (PyTorch Image Models) ecosystem and the Hugging Face model hub. By encapsulating model instantiation, parameter freezing, and dimensional introspection within a singular factory pattern, the architecture ensures that downstream components remain entirely agnostic to the underlying feature extractor's internal topology.

11.1.1 Candidate Foundation Models

The factory systematically supports a curated portfolio of candidate backbones, each occupying a distinct position on the Pareto frontier of accuracy, inference latency, and computational cost. The dynamic registry maps canonical model identifiers to explicit weights:

- **DINOv3 (`vit_small_patch16_dinov3`)**: Representing the state-of-the-art in self-supervised vision transformer foundation models, DINOv3 extracts dense morphological representations without relying on explicit taxonomic supervision. The architecture employs dedicated register tokens that eliminate high-norm artefacts in smooth background regions, thereby preserving fine-grained morphological features such as insect wing venation. It yields a compact 384-dimensional dense feature vector, facilitating highly efficient vector database indexing.
- **BioCLIP-2 (`hf-hub:imageomics/bioclip-2`)**: This domain-adapted vision-language model incorporates robust biological priors, having been contrastively pre-trained across vast swathes

of the Tree of Life. With approximately 86 million parameters, it maps visual morphology directly to a 512-dimensional semantic embedding space. While excelling in zero-shot and few-shot contexts on specialised taxa, its physical footprint necessitates strict memory management.

- **DINOv2 (`vit_small_patch14_dinov2.lvd142m`)**: A predecessor to DINOv3, DINOv2 remains a highly dense visual baseline. Utilising a ViT-Small architecture with a patch size of 14, it similarly produces a 384-dimensional representation and serves as a highly robust general-purpose foundational prior for comparative baseline benchmarking.
- **MobileNetV4 (`mobilenetv4_conv_small.e2400_r224_in1k`)**: Engineered specifically for ultra-low latency edge deployment and offline environments, this universal inverted bottleneck architecture contains merely 3.8 million parameters. Emitting a 960-dimensional pooled feature vector, it ensures tight sub-20 ms inference bounds on standard CPU cores, completely bypassing the memory and latency penalties associated with heavy self-attention mechanisms.

11.1.2 Parameter Freezing and Dimensional Introspection

Upon instantiation via the `create_feature_extractor` routine, the factory systematically removes the classification head (`num_classes=0`), returning dense pooled feature embeddings rather than categorical logits. Furthermore, the routine enforces strict parameter freezing across the entire backbone by setting `requires_grad = False` and shifting the module into deterministic evaluation mode (`eval()`). This physical isolation completely halts gradient tracking and stochastic operations (such as dropout or batch normalisation running statistics updates), forming the structural prerequisite for the decoupled training topology.

Crucially, the factory dynamically resolves the output feature embedding dimension via the `get_feature_dimension` function. Rather than relying on hardcoded configurations, the system executes a deterministic single-sample forward pass using a zero-initialised dummy tensor. The dimensions of this dummy tensor are injected from the centralised `taxon_vision.config` schema, ensuring that the resolved dimensionality (e.g. 384, 512, or 960) remains absolutely synchronised with the instantiated weights. This exact feature dimension is subsequently utilised to parameterise the downstream linear classification head and configure the traversal depth of the Hierarchical Navigable Small World (HNSW) retrieval graphs.

11.2 Feature Extraction and Embedding Pre-Computation

The standard supervised training paradigm of executing end-to-end backpropagation across deep neural networks during every training epoch is mathematically redundant when fine-tuning heavily parameterised foundation models for new classification tasks. Propagating gradients through over 100 million parameters simply to update a lightweight terminal linear layer wastes immense computational resources and heavily constrains the speed of hyperparameter iteration.

11.2.1 The Decoupled Extraction Paradigm

To circumvent this severe inefficiency, the TaxonVision architecture employs a mathematically rigorous two-stage methodology. The first stage consists of a pure forward-pass feature extraction phase. The entire dataset is passed through the frozen vision foundation model exactly once. The resulting dense, high-dimensional representations are extracted and persisted to storage.

The `extract_features` function is explicitly wrapped in the `torch.inference_mode()` context manager. This execution mode bypasses the PyTorch autograd engine entirely, suppressing the allocation of gradient buffers and the construction of the dynamic computational graph. By eliminating these graph-building overheads, tensor allocations remain strictly bound to the immediate activation memory, substantially elevating the maximum permissible batch size and optimising GPU memory throughput.

11.2.2 Batch Extraction Mechanics and Tensor Normalisation

During the extraction phase, the images are strictly normalised using channel-wise standardisation factors derived from the pre-training distribution (typically ImageNet statistics, configured via the application schema). The extraction loop systematically iterates over the `DataLoader`, projecting each 4D image batch tensor, denoted as $X \in \mathbb{R}^{B \times C \times H \times W}$, into a 2D representation tensor $Z \in \mathbb{R}^{B \times D}$, where D represents the backbone's feature dimension.

The resultant activation tensors are immediately detached from the active device and transferred to host CPU memory. This continuous offloading strictly prevents the exhaustion of precious GPU VRAM, allowing the architecture to process datasets scaling to millions of biological observations on heavily constrained bare-metal hardware.

11.3 Persistent Embedding Caching System

Once extracted, the gigabyte-scale arrays of feature embeddings and their associated ground-truth taxonomic labels must be reliably persisted and efficiently loaded during subsequent training invocations. The `src/taxon_vision/models/training/embeddings.py` module dictates the mathematical and physical invariants governing this caching engine.

11.3.1 On-Disk Serialisation and Key Generation

The integrity of the cache is paramount; if the caching system erroneously serves stale representations that do not precisely correspond to the current dataset state or backbone architecture, the resulting model will suffer catastrophic catastrophic divergence. To enforce absolute mathematical reproducibility, the caching system implements a cryptographically secure invalidation invariant.

Cache files are stored on disk with a uniquely generated hash key. This deterministic cache key is synthesised by hashing a strictly ordered concatenation of critical parameters:

1. The precise canonical identifier of the active foundational backbone (e.g., `vit_small_patch16_dinov3`).
2. A cryptographic hash (MD5 or SHA-256) of the underlying dataset manifest, specifically tracking the DVC dataset versioning file, thereby detecting any alterations to the constituent images or labelling matrices.
3. The serialised parameters of the image transformation pipeline, particularly the spatial resolution (224×224) and normalisation constants, ensuring that alterations to the input pre-processing immediately invalidate the historical cache.

If any of these invariants diverge, the system gracefully bypasses the cache and forces a full re-extraction over the active hardware.

11.3.2 Memory-Mapped Tensor Loading

Loading massive datasets into memory natively poses severe engineering challenges. A dataset comprising 500,000 biological observations projected into a 768-dimensional FP32 space consumes approximately 1.5 gigabytes of memory purely for the embedding vectors, excluding labels and runtime overhead. Attempting to allocate these contiguous arrays entirely within the system RAM can rapidly trigger out-of-memory (OOM) faults, particularly during parallel hyperparameter tuning trials executed via Optuna.

To guarantee operational stability, the caching engine inherently supports memory mapping via `mmap_mode`. By writing the contiguous NumPy or PyTorch tensors to disk using a memory-mapped format, the operating system's virtual memory manager is permitted to treat the file on disk as though it were resident in RAM. When the PyTorch `DataLoader` subsequently requests a specific mini-batch permutation during the `_train_cached_epoch` execution, the operating system transparently pages only the specifically requested memory chunks into the active physical memory.

This memory-mapping architecture explicitly severs the correlation between the absolute dataset size and the peak RAM allocation, allowing the TaxonVision pipeline to train robust linear heads on datasets that vastly exceed the physical memory constraints of the underlying node, whilst maintaining high-throughput sequential reads.

11.4 Throughput and Resource Benchmarking

The mathematical decoupling of feature extraction from gradient optimisation fundamentally transforms the computational complexity of the training lifecycle, yielding radical accelerations in iteration latency.

11.4.1 Complexity and Speedup Analysis

Consider an architecture where a standard epoch involves computing a forward pass, loss calculation, backward pass, and weight update over the complete dataset. For a foundation model comprising P parameters (where $P > 100 \times 10^6$ for large ViT models), the floating-point operations (FLOPs) required per epoch are heavily dominated by the backbone matrix multiplications.

By caching the representations, the computational complexity of the subsequent classification head training collapses. The gradient backpropagation is strictly restricted to the terminal linear layer. The new time complexity per epoch is fundamentally bound by $\mathcal{O}(N \times D \times K)$, where N is the number of samples, D is the feature dimension, and K is the number of taxonomic classes.

Empirical benchmarking of the TaxonVision platform on a bare-metal NVIDIA RTX 5070 Ti demonstrates that executing a single training epoch through an end-to-end BioCLIP-2 architecture requires multiple minutes and entirely saturates the GPU VRAM. Conversely, iterating over the pre-computed, memory-mapped embedding split requires less than two seconds per epoch. This translates to an operational speedup exceeding 50×, empowering data scientists to execute exhaustive Bayesian hyperparameter searches across hundreds of trials within the time previously required to complete a singular traditional training run.

Furthermore, because the backbone remains entirely dormant during head optimisation, the static memory footprint of the GPU drops drastically. This ensures that the training pipeline easily complies with the strict 70% `TAXON_CUDA_MEMORY_FRACTION` limit, perfectly preserving the remaining 30% of the memory pool for zero-downtime serving and live inference queries under concurrent workloads.

Chapter 12

Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics

12.1 The Biological Long-Tail Pareto Distribution

12.2 Effective Number of Samples Mathematical Derivation

12.3 PyTorch Implementation and Numerical Stabilisation

Chapter 13

Classification Head Architecture, Optimization Loops, and Callbacks

13.1 Classification Head Geometry and Transfer Learning

13.2 Cached Embedding Training Loop and Mini-Batch Dynamics

13.3 Convergence Callbacks and Best-Weight Restoration

Chapter 14

Bayesian Hyperparameter Optimization with Optuna TPE and ASHA

14.1 Bayesian Optimization Principles and TPE Mechanics

14.2 Search Space Parameterisation and Objective Formulation

14.3 Asynchronous Successive Halving (ASHA) Pruning in Practice

Chapter 15

Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry

15.1 Experiment Tracking Architecture on DagsHub

15.2 Deterministic Lifecycle Management via RAII Scopes

15.3 Cryptographic Lineage Capture and Automated Model Promotion

Chapter 16

Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection

16.1 The Epistemic Safety Problem in Biological Vision

16.2 Split Conformal Prediction and Statistical Coverage Bounds

16.3 Helmholtz Free Energy Out-of-Distribution Detection

Chapter 17

Active Learning Strategy and Priority Triage Queue

17.1 Uncertainty-Based Query Acquisition Mechanics

17.2 Priority Triage Queue Implementation

17.3 Closing the Continuous Retraining Annotation Loop

Chapter 18

ONNX Export, Zero-Allocation Runtime, and INT8 Quantization

18.1 Computational Graph Export and Dynamic Axes

18.2 Zero-Allocation C++ Inference Execution Engine

18.3 INT8 Quantization Calibration and Class Activation Mapping

Chapter 19

FastAPI REST Service and Server-Rendered HTMX Operator Dashboard

19.1 Unified REST and Web Application Architecture

19.2 API Schemas, Endpoints, and Inference Routing

19.3 Server-Rendered HTMX Operator Interface

Chapter 20

Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection

20.1 Prometheus Telemetry Instrumentation and Metrics Design

20.2 Serving Latency and Conformal Set Monitoring

20.3 Embedding Representation Drift via Wasserstein and MMD

Chapter 21

Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets

21.1 Testing Architecture and Test Matrix Topology

21.2 Mathematical Invariant Assertions and Property-Based Testing

21.3 Cognitive Complexity Budgets and Pre-Commit Quality Gates

Chapter 22

Open Knowledge Format (OKF v0.2) and Master Agent Routing

22.1 Open Knowledge Format (OKF v0.2) Schema and Trust Tiers

22.2 Multi-Agent Governance and Decision Routing Matrices

22.3 Automated Validation Scripts and D3 Knowledge Graph Visualization

Chapter 23

Developer Operations, Environment Setup, and Engineering Runbooks

23.1 Developer Workstation Onboarding and Environment Setup

23.2 Dataset Synchronization and Remote Storage Runbook

23.3 Cluster Operations, GPU Debugging, and Emergency Rollback

Bibliography

[1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[2] Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse gradient embeddings. In *International Conference on Learning Representations (ICLR)*, 2020.

[3] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9268–9277. IEEE, 2019.

[4] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xinwang Wang. A survey of deep active learning. *ACM Computing Surveys (CSUR)*, 54(9):1–40, 2021.

[5] Samuel Stevens, Jiaman Wu, Matthew J. Thompson, Elizabeth G. Campolongo, Chan Hee Song, David E. Carlyn, Li Dong, Wasila M. Dahdul, Charles Stewart, Tanya Berger-Wolf, Wei-Lun Chao, and Yu Su. Bioclip: A vision foundation model for the tree of life. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19412–19424. IEEE, 2024.

[6] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The inaturalist species classification and detection dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8769–8778. IEEE, 2018.